Prompt Engineering Handbook

A guide to the Prompt Recipe Book: structure, categories, optimization, testing and responsible use of AI prompts

Prepared by **Likith G**

Minor Project 4 | AI Engineering | Launched Global

Contents

Contents 2

Introduction 4

What prompt engineering means 4

Why prompt quality matters 4

Purpose of the Prompt Recipe Book 4

Who can use the library 4

Prompt Structure 6

The six elements in detail 6

Not every prompt needs all six elements 7

Anatomy of a prompt 7

Prompt Categories 9

Category 1: Content Creation 9

Category 2: Coding Assistance 9

Category 3: Research and Analysis 9

Category 4: Productivity Workflows 10

Design patterns shared across the library 10

Examples from the library 11

Prompt Optimization 12

Step 1: Identify vague instructions 12

Step 2: Improve the prompt 12

Step 3: Use test inputs 13

Step 4: Compare original and revised versions 13

Testing and Evaluation 15

Evaluation criteria 15

Scoring scale 15

Evaluation procedure 16

Score recording template 16

Limitations of this evaluation 16

Responsible AI Usage 17

1. Verify factual claims . . . 17
2. Avoid fabricated sources . . . 17
3. Protect personal and confidential information . . . 17
4. Review generated code before use . . . 17
5. Check outputs for bias and errors . . . 18
6. Follow copyright and academic-integrity requirements . . . 18

Conclusion . . . 19

What was learned . . . 19
How the prompt library could be expanded . . . 19
Closing note . . . 19

Appendix: Prompt Library Index . . . 21

CHAPTER 1

Introduction

What prompt engineering means

A **prompt** is the set of instructions and information given to an AI model. **Prompt engineering** is the practice of designing, testing and refining those instructions so that the model produces output that is useful, accurate and in the form you need. It is closer to writing a clear brief for a skilled colleague than to typing a search query: the more precisely the task, audience, limits and expected result are described, the less the model has to guess.

Prompt engineering is not about finding magic words. It is a repeatable process of stating what you want, checking what you get, and improving the instructions based on evidence.

Why prompt quality matters

- **Relevance.** Vague prompts produce generic answers. A prompt that names the audience, purpose and format produces output that can be used with little editing.
- **Accuracy and honesty.** Language models can produce confident but unsupported statements. Instructions such as “do not invent citations” or “use only information supported by the supplied text” reduce this risk, though they do not remove it.
- **Consistency.** A well-structured prompt gives similar quality across repeated uses, which matters when a prompt is reused by many people.
- **Efficiency.** Time spent on one good prompt saves repeated rounds of correction later.
- **Safety.** Clear constraints help keep outputs within what the user can verify and responsibly use.

Purpose of the Prompt Recipe Book

The **Prompt Recipe Book** is a reusable library of 20 tested prompt templates, organized into four categories of five prompts each: Content Creation, Coding Assistance, Research and Analysis, and Productivity Workflows. Each template uses clearly marked placeholders in square brackets, such as [TOPIC] or [PASTE CODE], so a user can fill in their own details and run the prompt immediately.

This handbook is the companion document. It explains *how* the prompts are designed, *why* they are written the way they are, how they were improved and evaluated, and how to use them responsibly.

Who can use the library

User	How the library helps
Students	Study planning, code explanation, research summaries, presentation outlines, and writing support for assignments (subject to academic-integrity rules).

User	How the library helps
Early-career professionals	Professional emails, LinkedIn posts, meeting summaries, task prioritization and weekly reviews.
Developers and learners	Code generation, debugging, code review and SQL query drafting, with beginner-friendly explanations.
Analysts	Structured comparisons, SWOT analysis, dataset interpretation and paper summarization.
Anyone new to AI tools	Ready-made starting points that demonstrate what a well-structured prompt looks like.

No programming or prompt-engineering background is required. The only skill needed is the ability to replace the bracketed placeholders with accurate details and to review the result critically.

CHAPTER 2

Prompt Structure

Most strong prompts in the library are built from up to six elements. Thinking in these elements gives a checklist for writing a new prompt and for diagnosing why an existing one is not working.

#	Element	Question it answers	Example from the library
1	**Role**	What expertise or perspective should the AI adopt?	“Act as a professional LinkedIn content strategist.”
2	**Objective**	What should the AI accomplish?	“Create a LinkedIn post about [PROJECT OR TOPIC].”
3	**Context**	What background does it need?	“The post should communicate what I learned, what I built, the challenges I encountered…”
4	**Instructions**	What specific actions should it take?	“Highlight three important learnings.”
5	**Constraints**	What requirements and limits apply?	“Do not invent achievements or results.”
6	**Output Format**	How should the answer be organized?	“Opening / Project Overview / Key Learnings / Conclusion / Hashtags”

The six elements in detail

1. Role

The role tells the model which expertise and viewpoint to use. “Act as a software debugging specialist” leads to a different style of answer than “Act as a programming instructor.” Choose a role that matches the task and the reader. A role sets perspective and tone; it does not make the model more factually reliable, so it should never replace verification.

2. Objective

The objective is a single clear statement of the outcome. Good objectives name the deliverable (“a comparison table”, “a study schedule”) rather than a vague activity (“help with studying”). If a prompt has several unrelated objectives, split it into separate prompts.

3. Context

Context supplies facts the model cannot know: the audience, the data, the code, the deadline, the platform or the situation. In the library, context is usually provided through placeholders such as [AUDIENCE], [SCHEMA] or [DEADLINE]. Missing context is the most common reason for generic output.

4. Instructions

Instructions are the concrete steps or components the answer must include. Numbered or bulleted instructions work well because they make the output easy to check: either each item is present or it is not. Example: the Code Debugging Assistant asks the model to identify the cause, explain why it occurs, provide the smallest necessary fix, explain the changes, and suggest test cases.

5. Constraints

Constraints set boundaries. In this library they mainly protect honesty and realism, for example “Do not claim the code has been executed unless it has,” “Use only the supplied schema,” and “Do not schedule more time than is available.” Constraints can also cover length, tone, reading level or topics to avoid.

6. Output Format

The output format tells the model how to organize the answer: headings, a table, a numbered list, or a subject line followed by an email body. A defined format makes responses easier to read, compare and reuse, and it makes testing easier because the expected structure is known in advance.

Not every prompt needs all six elements

The structure should fit the task. A simple prompt with a clear purpose may only need a role, an objective and one or two constraints. Adding every element to every prompt makes prompts long and harder to maintain, and extra instructions can conflict with each other. The library therefore uses the elements in different combinations:

Prompt	Elements used (and how)
LinkedIn Post Generator (Content Creation)	All six, each labelled with a heading (ROLE, OBJECTIVE, CONTEXT, REQUIREMENTS, OUTPUT FORMAT, TONE). Suited to a task where voice and structure both matter.
Social Media Caption Generator	Role, objective, context (platform, audience, tone placeholders), a short list of constraints, and a format (“numbered list”). No separate background section is needed.
Task Prioritization Assistant	Role, context (the task list), criteria to consider, an output grouping (High / Medium / Low) and an instruction to flag unclear tasks.
Code Explanation Assistant	Role, context (the code), a list of required components, and a style constraint (“plain English, avoid unnecessary jargon”).

Anatomy of a prompt

The following excerpt from the Meeting Summary Generator shows how the elements combine even without explicit labels.

ANNOTATED EXAMPLE

```
Act as a meeting documentation assistant.            <- Role
Meeting notes:                                        <- Context
[PASTE NOTES]
Create a structured summary containing:               <- Objective + Instructions
- Main discussion points.
- Decisions made.
- Action items.
- Unresolved questions.
Do not invent decisions, owners, or deadlines.        <- Constraints
Mark missing information as unspecified.
```

Quick test for a new prompt

Ask: Would a capable person who knew nothing about my situation be able to do this task from the prompt alone? If they would need to ask a question first, the answer to that question probably belongs in the prompt as context, an instruction or a constraint.

CHAPTER 3

Prompt Categories

The library contains 20 prompts in four categories. The categories were chosen because they cover the everyday tasks of a student or early-career professional working in business, data and technology.

Category 1: Content Creation

Prompts for producing written and presentation material for a defined audience. Their shared constraint is that the AI must not invent facts, achievements or commitments.

Prompt	Purpose
Professional LinkedIn Post Generator	Draft an authentic post about a project, technology or learning, with an opening, overview, three learnings and hashtags.
Blog Article Generator	Write a beginner-friendly article with headings, three practical examples, common mistakes and five takeaways.
Social Media Caption Generator	Produce five distinct caption options for a given platform, audience and tone.
Professional Email Drafting	Draft an email with a subject line, a clear purpose and a next step, without inventing facts.
Presentation Outline Generator	Create a slide-by-slide outline with titles, key points, suggested visuals and speaker notes.

Category 2: Coding Assistance

Prompts for writing, fixing, explaining and reviewing code and SQL. Their shared constraint is honesty about execution: the model must not claim code has run or tests have passed unless that is actually true.

Prompt	Purpose
Code Generator	Build a beginner-friendly solution with comments, error handling, run instructions and basic test cases.
Code Debugging Assistant	Find the likely cause of a bug, explain it, give the smallest necessary fix and suggest tests.
Code Explanation Assistant	Explain code step by step in plain English with an example and two common beginner mistakes.
SQL Query Generator	Write a query from a schema, explain each clause, note NULL issues and suggest a way to validate results.
Code Review Assistant	Review correctness, readability, maintainability, error handling, performance and security; separate confirmed issues from possible concerns.

Category 3: Research and Analysis

Prompts for understanding topics, comparing options and interpreting information. Their shared constraint is the separation of verified facts from assumptions, and no invented sources or numbers.

Prompt	Purpose
Research Topic Explainer	Structured explanation with background, concepts, findings, viewpoints, limitations and further reading.
Comparative Analysis	Compare two options against stated criteria in a table, with trade-offs and when each is more suitable.
Research Paper Summarizer	Summarize objective, question, method, findings, limitations and implications using only the supplied text.
Business Data Analyst	Identify trends and unusual observations, suggest visualizations and three business actions, and state data limitations.
SWOT Analysis Generator	Produce a reasoned SWOT, distinguishing known facts from assumptions, with three strategic recommendations.

Category 4: Productivity Workflows

Prompts for planning and organizing work. Their shared constraint is realism: plans must fit the available time and must not invent deadlines, owners or decisions.

Prompt	Purpose
Study Planner	Create a realistic schedule with study blocks, revision, practice, breaks and weekly reviews.
Task Prioritization Assistant	Group tasks by priority using deadline, importance, effort and dependencies; flag unclear tasks.
Meeting Summary Generator	Summarize discussion, decisions, action items and open questions; mark missing details as unspecified.
Weekly Review Assistant	Review accomplishments, unfinished work, obstacles and lessons; set priorities and three improvements.
Project Breakdown Assistant	Split a project into phases with tasks, deliverables, dependencies, effort, completion criteria and risks.

Design patterns shared across the library

- **Bracketed placeholders** such as [TOPIC] or [PASTE CODE] show exactly what the user must supply.
- **Anti-fabrication constraints** appear in nearly every prompt (“Do not invent…”, “Use only the supplied…”).
- **Facts versus assumptions.** Research prompts ask the model to state assumptions and missing information.
- **Explicit structure.** Numbered tasks or named sections make each response easy to check.

- **Beginner-friendly defaults.** Several coding prompts set the skill level to Beginner and ask for plain English.

Examples from the library

CODING ASSISTANCE: SQL QUERY GENERATOR (EXCERPT)

```
Act as a SQL developer and data analyst.
Task: [DESCRIBE THE DATA QUESTION].
Database system: [MYSQL / POSTGRESQL / BIGQUERY / OTHER].
Available tables and columns: [SCHEMA].
Requirements:
- Write the SQL query.
- Explain each important clause.
- State any assumptions.
- Identify potential NULL-value issues.
- Suggest a way to validate the results.
Use only the supplied schema.
```

RESEARCH AND ANALYSIS: SWOT ANALYSIS GENERATOR (EXCERPT)

```
Act as a strategic business analyst.
Prepare a SWOT analysis for [BUSINESS OR PROJECT].
For each category, explain the reasoning.
Separate known facts from assumptions.
Identify missing information that could change the analysis.
Finish with three practical strategic recommendations.
```

PRODUCTIVITY WORKFLOWS: STUDY PLANNER (EXCERPT)

```
Act as a study planning assistant.
Subjects: [SUBJECTS].
Available study time: [HOURS PER DAY].
Deadline: [DATE].
Create a realistic study schedule.
Do not schedule more time than is available.
```

CHAPTER 4

Prompt Optimization

Prompt optimization is the cycle of drafting a prompt, testing it on realistic inputs, finding where the output falls short, and revising the wording. Each revision should be driven by an observed problem, not by guesswork.

Step 1: Identify vague instructions

Each draft prompt was read line by line, looking for instructions that two different people could interpret in different ways. Common warning signs:

Warning sign	Why it causes problems	Typical fix
Subjective words: “good”, “engaging”, “professional”	The model must guess the standard being applied.	Define it: name the tone, audience and what “engaging” means (for example, a clear opening line).
No audience stated	Output defaults to a generic reader and level.	Add an audience placeholder such as [AUDIENCE].
No output format	Structure varies between runs and is hard to compare.	Specify sections, a table or a numbered list.
Unbounded quantity: “some examples”	Length and depth are unpredictable.	Give counts: “three practical examples”, “five key takeaways”.
No rule about missing information	The model may fill gaps with invented details.	Add “Do not invent…” and “Mark missing information as unspecified”.
Several tasks in one sentence	Some tasks are skipped or merged.	Break into numbered tasks.

Step 2: Improve the prompt

Improvements followed a fixed order so that the effect of each change could be understood:

1. **Clarify the objective** so that it names a concrete deliverable.
2. **Add missing context** through placeholders for audience, data, platform or deadline.
3. **Convert prose into numbered instructions** so each required component can be checked.
4. **Add constraints** that address observed failure modes, such as invented facts or unrealistic schedules.
5. **Define the output format** with named sections, tables or lists.
6. **Trim** any instruction that did not change the output, to keep the prompt short.

Worked example: Meeting Summary Generator

The following is an **illustrative** example of the technique, showing a typical first draft and the revised library version. It demonstrates the method; it is not a recorded test result.

ORIGINAL DRAFT (ILLUSTRATIVE)

```
Summarize these meeting notes: [PASTE NOTES]
```

Problems spotted: no definition of “summarize”; no structure; no rule for missing owners or deadlines, so the model may invent them.

REVISED VERSION (FROM THE LIBRARY)

```
Act as a meeting documentation assistant.
Meeting notes: [PASTE NOTES]
Create a structured summary containing:
- Main discussion points.
- Decisions made.
- Action items.
- Assigned owners, if explicitly stated.
- Deadlines, if explicitly stated.
- Unresolved questions.
Do not invent decisions, owners, or deadlines.
Mark missing information as unspecified.
```

Step 3: Use test inputs

Every prompt was run against a small, deliberate set of test inputs rather than a single convenient one. Three types of input were used:

Input type	Purpose	Example for the Meeting Summary prompt
Typical	Confirms the prompt works in the normal case.	Notes with clear decisions, named owners and dates.
Incomplete	Checks how the prompt handles missing information.	Notes where no owner or deadline is mentioned. Expected: marked as “unspecified”.
Edge case	Probes the limits of the instructions.	Very short notes, or notes that contain no decisions at all.

Using the same inputs for every version of a prompt is essential: if the inputs change between tests, differences in results cannot be attributed to the prompt.

Step 4: Compare original and revised versions

To compare versions fairly, the original and revised prompts were run on identical inputs, using the same AI model and settings, and the outputs were scored against the same criteria (see Chapter 5). Changes were made one group at a time where possible so that an improvement could be linked to a specific edit.

Comparison step	What is recorded
1. Run original prompt on each test input	Output saved with the prompt version number.
2. Run revised prompt on the same inputs	Output saved with the new version number.
3. Score both outputs using the rubric	Scores per criterion, plus short notes on what went wrong or right.

Comparison step	What is recorded
4. Compare scores and read the outputs side by side	Which criteria improved, stayed the same or got worse.
5. Keep, revise again, or revert	Decision and reason logged in the change history.

Reading the results

A higher score does not automatically mean a better prompt. Always read the actual outputs: a revision can raise the format score while quietly lowering accuracy. If a change does not improve any criterion, remove it.

CHAPTER 5

Testing and Evaluation

Evaluation turns “this seems better” into something that can be recorded and compared. The scoring system below is intentionally simple so that it can be applied consistently by one person without special tools.

Evaluation criteria

Criterion	What it measures	Questions to ask
Relevance	Does the output do the task that was asked?	Is every requested component present? Is the content on topic for the audience?
Accuracy and honesty	Is the output free of invented or unsupported content?	Are there fabricated facts, sources, numbers or claims of execution? Does it flag uncertainty?
Instruction following	Does it respect the stated constraints?	Were counts, tone, length and “do not” rules obeyed?
Structure and format	Does it match the requested output format?	Are the sections, table or list present and in the right order?
Clarity and usability	Could the output be used with minimal editing?	Is it readable, well-organized and appropriate for a beginner?
Consistency	Is quality stable across test inputs and repeated runs?	Do the typical, incomplete and edge-case inputs all give acceptable results?

Scoring scale

Each criterion is scored from 1 to 5, giving a total out of 30.

Score	Meaning
5	Fully meets the criterion; no changes needed.
4	Meets the criterion with minor issues.
3	Partly meets the criterion; noticeable gaps.
2	Largely fails the criterion; significant problems.
1	Fails the criterion or the output is unusable on this point.

Total (out of 30)	Interpretation
26 - 30	Ready for the library.
20 - 25	Usable; revise the lowest-scoring criteria.
Below 20	Needs substantial redesign.

Accuracy and honesty is treated as a gating criterion: a prompt that scores 1 or 2 on it should be revised regardless of its total, because fabricated content can cause real harm even if everything else is excellent.

Evaluation procedure

1. Choose the prompt and fill in the placeholders for each test input (typical, incomplete, edge case).
2. Run the prompt with the same model and settings each time, and save the full output.
3. Score each output against the six criteria before looking at other versions, to reduce bias.
4. Average the scores across test inputs to get the prompt's result.
5. Record the score, the model used, the date and notes on weaknesses in the Prompt Performance Report.

Score recording template

Prompt	Test input	Rel.	Acc.	Instr.	Format	Clarity	Consist.	Total /30
	Typical							
	Incomplete							
	Edge case							
	Average							

Limitations of this evaluation

- **Subjectivity.** Scores come from human judgment. A single rater can be inconsistent or biased towards a favoured version. Using a second reviewer improves reliability.
- **Small sample.** A few test inputs per prompt cannot cover every real-world situation, so a high score does not guarantee good performance on all inputs.
- **Non-deterministic outputs.** The same prompt can give different answers on different runs. A single run may be better or worse than typical.
- **Model dependence.** Results apply to the model and version tested. A different model, or an updated version of the same one, may behave differently.
- **Limited fact-checking.** The rubric checks for signs of fabrication, but verifying every factual claim still requires independent sources.
- **Tool access.** Prompts that need current information, such as the Research Topic Explainer, cannot be fully evaluated without a browsing-enabled tool.
- **Equal weighting.** The criteria are scored equally, although in practice accuracy may matter more than format for some tasks.

CHAPTER 6

Responsible AI Usage

AI-generated output is a draft, not a finished or guaranteed-correct product. The person who uses it remains responsible for its accuracy, legality and consequences. The following practices apply to every prompt in the library.

1. Verify factual claims

Language models can state incorrect information fluently and with confidence. Check important facts, dates, figures and definitions against reliable primary or authoritative sources before relying on them. Treat statistics and anything that will affect a decision with extra care. Prompts in the library ask the model to separate verified information from uncertainty, but that statement is itself generated text and must be checked.

2. Avoid fabricated sources

Models can produce references that look real but do not exist, or attribute statements to the wrong author. Never cite a source you have not opened and read. If the model suggests a book, paper or link, search for it independently to confirm it exists and says what is claimed. Research prompts in the library include the instruction “Do not invent citations or research findings,” and the Research Topic Explainer notes that, without a browsing-enabled tool, current sources may be unavailable.

3. Protect personal and confidential information

- Do not paste passwords, financial details, identity numbers, medical records or private messages into an AI tool.
- Remove names, contact details and identifying information from meeting notes, datasets and documents before using them in prompts such as the Meeting Summary Generator or Business Data Analyst.
- Do not share confidential employer, client or university material unless you are explicitly permitted to.
- Check the privacy settings and data policy of the tool you use, and prefer anonymized or sample data when testing.

4. Review generated code before use

- Read and understand the code before running it. If you cannot explain what it does, use the Code Explanation Assistant first.
- Run it in a safe test environment, not on production systems or important data.
- Test it yourself. The coding prompts forbid claiming that tests passed unless they were actually run, so run the suggested test cases and add your own.
- Check for security issues such as hard-coded secrets, unsafe handling of user input and unnecessary permissions, and review any third-party packages it suggests.

- For SQL, validate results against a known sample and test on a copy of the data before running anything that modifies it.

5. Check outputs for bias and errors

Models reflect patterns in their training data and can produce stereotyped, one-sided or culturally narrow content. Review output for unfair assumptions about people or groups, missing viewpoints and loaded wording. Also check basic errors: arithmetic, inconsistent details, wrong names, and instructions that were silently ignored. Asking the model to list assumptions and different viewpoints, as several library prompts do, makes such problems easier to spot.

6. Follow copyright and academic-integrity requirements

- **Copyright.** Do not paste large portions of copyrighted material into prompts or republish AI output that reproduces protected work. Credit and cite sources when using their ideas or text.
- **Academic integrity.** Follow your institution’s policy on AI tools. Where AI assistance is permitted, use it to support your learning, such as planning, explaining and reviewing, rather than to submit work that is not your own. Disclose AI use when required.
- **Authenticity.** For professional content such as LinkedIn posts, only claim achievements and skills that are genuinely yours. The Content Creation prompts include “Do not invent achievements or results” for this reason.

Pre-use checklist

Before using any AI output: (1) Have I verified the key facts? (2) Have I confirmed every source exists? (3) Did I avoid sharing sensitive data? (4) Have I tested and understood any code? (5) Have I checked for bias and errors? (6) Does my use comply with copyright and my institution’s rules?

CHAPTER 7

Conclusion

What was learned

- **Clarity beats cleverness.** The largest quality gains come from specifying audience, task, constraints and format, not from unusual wording.
- **Structure helps.** The six-element structure (role, objective, context, instructions, constraints, output format) works as a design checklist and a debugging tool.
- **Constraints protect honesty.** Explicit rules such as “do not invent” and “mark missing information as unspecified” make output safer, though they do not replace human verification.
- **Testing is necessary.** A prompt that works once may fail on incomplete or unusual input. Typical, incomplete and edge-case tests reveal weaknesses early.
- **Measurement needs discipline.** Fair comparison requires identical inputs, a consistent rubric and attention to the limits of the method.
- **Responsibility stays with the user.** AI can speed up drafting, planning and analysis, but accuracy, ethics and compliance still depend on human review.

How the prompt library could be expanded

Direction	Idea
New categories	Data analysis and visualization, customer support, career development (resumes, interview practice), learning and revision (flashcards, quizzes).
More prompts per category	Variants for specific platforms (Instagram, X), languages (Python, JavaScript, R), or industries.
Version control	Store prompts in a repository with version numbers, a change log and a record of which test inputs were used.
Multi-model testing	Evaluate the same prompts across several AI models and record differences.
Better evaluation	Add a second reviewer, more test inputs, weighted criteria, and repeated runs to measure consistency.
Richer prompting techniques	Add worked examples inside prompts, step-by-step reasoning requests for complex tasks, and prompt chains for multi-stage workflows.
Usability improvements	Add a short “How to use” note and a sample input and output for each prompt, plus a searchable index.
Community feedback	Collect user feedback on which prompts work and where they fail, and feed it into revisions.

Closing note

Prompt engineering is an iterative skill. A prompt library is never finished: models change, needs evolve and testing reveals new weaknesses. By keeping prompts structured, testing them honestly and using the results responsibly, the Prompt Recipe Book can remain a reliable and growing resource.

CHAPTER APPENDIX

Appendix: Prompt Library Index

Category	No.	Prompt
Content Creation	1	Professional LinkedIn Post Generator
Content Creation	2	Blog Article Generator
Content Creation	3	Social Media Caption Generator
Content Creation	4	Professional Email Drafting
Content Creation	5	Presentation Outline Generator
Coding Assistance	1	Code Generator
Coding Assistance	2	Code Debugging Assistant
Coding Assistance	3	Code Explanation Assistant
Coding Assistance	4	SQL Query Generator
Coding Assistance	5	Code Review Assistant
Research and Analysis	1	Research Topic Explainer
Research and Analysis	2	Comparative Analysis
Research and Analysis	3	Research Paper Summarizer
Research and Analysis	4	Business Data Analyst
Research and Analysis	5	SWOT Analysis Generator
Productivity Workflows	1	Study Planner
Productivity Workflows	2	Task Prioritization Assistant
Productivity Workflows	3	Meeting Summary Generator
Productivity Workflows	4	Weekly Review Assistant
Productivity Workflows	5	Project Breakdown Assistant